

Agent Design Documentation

Invoice Risk Evaluator — UiPath Agent Builder

This document records the design of the AI agent used in the invoice processing pipeline, since the agent itself lives in UiPath Studio Web / Orchestrator and cannot be exported as a portable file the way the rest of the project's code can. It captures the exact prompts, the input/output contract, and the reasoning behind the configuration choices below.

Purpose

Given a new invoice and that vendor's recent transaction history, decide whether the invoice amount is consistent with the vendor's normal pattern or represents an anomaly significant enough to warrant human review — replacing a static "amount over \$X" threshold with reasoning grounded in actual vendor behavior.

System Prompt

You are an invoice risk evaluator for a boxing equipment wholesale company. You receive the data of a new invoice and the historical invoices of that same vendor. Your job is to determine whether the current invoice's amount is consistent with that vendor's historical pattern, or whether it represents an anomaly that warrants human review. Classify the risk as Low, Medium, or High, explain your reasoning in one sentence, and decide whether this invoice requires human approval (true/false).

IMPORTANT: requiresReview MUST be set to true whenever riskLevel is "High". requiresReview may be false only when riskLevel is "Low" or "Medium".

Note: the second paragraph was added after observing that the model did not always set requiresReview consistently with its own riskLevel classification — see “Design Rationale” below.

User Prompt (template)

Evaluate the following invoice:

- Invoice ID: {{invoiceId}}
- Vendor: {{vendor}}
- Amount: {{amount}}
- Invoice Date: {{invoiceDate}}

Here is the historical data for this vendor:

{{vendorHistory}}

Based on this vendor's historical pattern, assess the risk level of this invoice.

Input Schema

Field	Type	Description
invoiceId	String	Unique invoice identifier
vendor	String	Vendor name
amount	Double	Invoice amount (Double, not Decimal — JSON Schema has no native decimal type; precision loss is immaterial for risk reasoning)
invoiceDate	String	Invoice date
vendorHistory	String	Raw JSON array of the vendor's 5 most recent invoices (least-context: not the full history)

Output Schema

Field	Type	Description
riskLevel	String	"Low" / "Medium" / "High"
reasoning	String	One-sentence natural-language explanation, grounded in the supplied vendor history
requiresReview	Boolean	Whether this invoice should be escalated to Action Center for human approval

Model & Parameters

Model: efficient / mid-tier model, not the largest available in its family

Temperature: 0.2 — this is a classification task that must be reproducible, not a generative one; the same invoice against the same history should return the same classification

Max output tokens: ~300–500 — the output is three small structured fields, not long-form text

Tools: none — see rationale below

Design Rationale

No Tools, by design. The agent always needs the vendor's history to reason at all — there is no case where that lookup is conditional. When an action is always required rather than situationally decided, resolving it deterministically in the surrounding workflow (an HTTP Request to Supabase before invoking the agent) is simpler and more predictable than letting the agent decide whether to call a tool for something it always needs anyway.

Deterministic guardrail in the surrounding workflow. In testing, the agent occasionally classified an invoice as High risk in its reasoning without setting requiresReview to true, despite the explicit instruction added to the system prompt. Rather than relying solely on prompt tuning to force 100% consistency from a non-deterministic model, the UiPath workflow itself checks riskLevel = "High" OR requiresReview before deciding whether to escalate — so a High classification always triggers Action Center review regardless of what the boolean output says.

Least-context principle. The agent receives only the vendor's 5 most recent invoices, not the full history. This keeps each call's input small and predictable in cost and latency, and in practice produced clearer, more specific reasoning than passing the entire history would have.

Double over Decimal for amount. JSON Schema — which defines the agent's input/output contract — has no native Decimal type, only number (Double) and integer. Since the agent only needs the magnitude of the amount to reason about anomalies, not to perform exact financial arithmetic, Double is the correct choice here

even though the rest of the UiPath workflow uses Decimal for the same value when it matters for exact calculations.